Advanced Machine Learning Project Proposal

Improving CLRNet Lane Detection Under Occlusion: Visibility Gating and Max-Pooled Channel Attention in ROIGather, with Occlusion-Consistency Training

Mahmoud Afifi Ziad Eliwa

1. Problem Statement and Motivation

Problem: A self-driving car must find the lane markings on the road from a front camera. This is hard because vehicles block the lines, markings can be worn or dashed, and shadows or sun can hide them.
Motivation: Lane detection is a safety-critical part of Advanced Driver Assistance Systems (ADAS). A good detector must use the whole scene to guess where a lane continues when other traffic hides it [1].
Our focus: We start from CLRNet [1], a strong detector with public code and weights. Occlusion is the hard case, so we test three small changes aimed at it:

1. a **visibility gate** inside ROIGather that turns down sampled points that are probably not on a visible marking,
2. a **max-pooled channel-attention block** (GMP-ECA) inside ROIGather,
3. an **occlusion-consistency loss** for training.

We compare them one by one and together on the TuSimple benchmark. We also measure robustness directly, with a fixed occlusion test at several masking levels (Section 13), and not only with clean accuracy or hand-picked pictures.

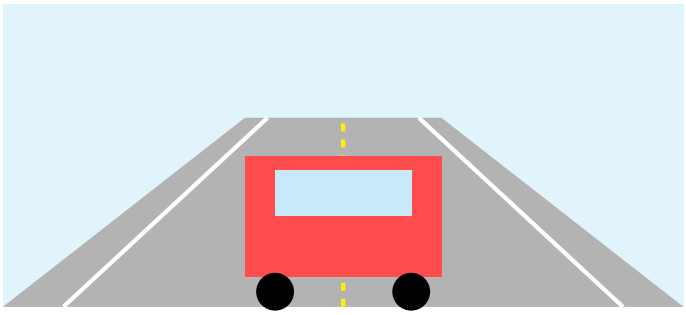


Figure 1: Problem example (schematic): the model must infer the lane even when a vehicle hides it.

2. Input/Output Examples and Format Alignment

Input: One RGB image (1280×720 pixels) from a car camera. CLRNet resizes it to 320×800.
Dataset labels: TuSimple labels are stored in JSON:

- `h_samples`: fixed row heights ($y \in [160, 170, \ldots, 710]$, 56 rows).
- `lanes`: for each lane, the horizontal position (x) at each row. Missing points are `-2`.

Model output: CLRNet predicts lanes from learnable line priors [1]. Each prior has:

1. a lane confidence score,
2. a start point (x, y), an angle θ and a length,
3. $N = 72$ horizontal offsets at evenly spaced rows.

Matching output to labels: At test time, CLRNet drops low-confidence priors (we use the repository's default threshold) and removes overlapping lanes with non-maximum suppression. The remaining lanes are mapped back to 1280×720, read off at the rows in `h_samples`, and rows outside the lane are set to `-2`. This gives a JSON file that the official evaluation script accepts.

Occluded inputs (robustness test): For the occlusion tests (Section 13.4) the input image has part of one lane painted over. The input size, output format and labels are the same. The ground truth is *not* changed, so the model must still output the hidden part of the lane.

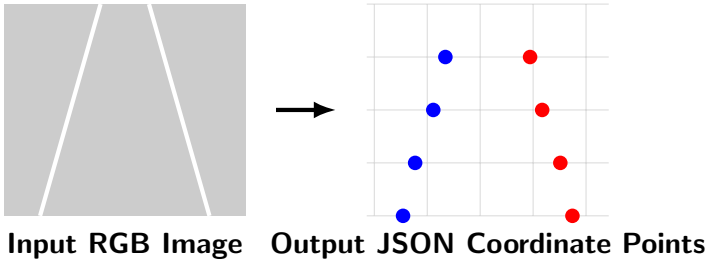


Figure 2: Lane outputs become horizontal positions at fixed rows (schematic; blue and red are two lanes).

3. Evaluation Metrics

We use the official TuSimple metrics:

- **Accuracy:** correct predicted points divided by ground-truth points, $\sum C_{\text{clip}} / \sum S_{\text{clip}}$. A point is correct if it is within 20 pixels of the ground truth. A lane counts as correct if at least 85% of its points are correct.
- **False Positive (FP):** wrong predicted lanes divided by all predicted lanes.
- **False Negative (FN):** missed ground-truth lanes divided by all ground-truth lanes.
- **F1 Score:** harmonic mean of precision $(1 - \text{FP})$ and recall $(1 - \text{FN})$.

In the occlusion tests only one lane per frame is hidden, so the official numbers, which average over all lanes in a frame, would dilute the effect. We therefore add two metrics that look only at the hidden lane. Both reuse the same point rule and lane rule as above.

- **Masked-lane recall:** among the ground-truth lanes that were occluded, the fraction the model still detects.
- **Masked-segment accuracy:** point accuracy computed only on the rows that lie under the mask.

4. Current State-of-the-Art Results

TuSimple is almost saturated: the top two results differ by 0.06 F1. These published numbers are all on the clean test set.

Method	Backbone	Acc (%)	F1 (%)	FP (%)	FN (%)	Source
Lane2Seq (CVPR 2024)	—	—	97.95	—	—	Reported by its author [3]
CLRNet (CVPR 2022)	ResNet-18	96.84	97.89	2.28	1.92	CLRNet paper, Table 3 [1]

Table 1: Best results we found on the TuSimple test set. "—" means not reported.

5. Dataset

We use the **TuSimple Lane Detection Benchmark** [5].

- **Link:** https://github.com/TuSimple/tusimple-benchmark
- **Content:** US highway driving clips in good and medium weather. Each clip has 20 frames and only the 20th is labeled. Labeled frames are 1280×720 RGB images with 2 to 5 lanes.
- **Splits:** 3,268 train, 358 validation and 2,782 test frames (6,408 total).
- **Size:** about 20GB for the raw download, under 2GB for the curated Kaggle version (Section 7).
- **Labels:** one JSON line per frame (values shortened below).

```
{"raw_file": "clips/<sequence>/<frame>/20.jpg",
 "h_samples": [160, 170, 180, ..., 700, 710],
 "lanes": [[-2, -2, ..., 632, 625, ..., 299],
           [-2, -2, ..., 811, 828, ..., 1148]]}
```

Figure 2 shows an input image and its output points.

6. Why TuSimple

- It is the standard lane detection benchmark, so we can compare with published results.
- CLRNet already has a data loader, a config and pretrained weights for it.
- It is small, so one training run is short and we can repeat runs with several random seeds. We need this because the gain we expect is small (Section 12).

Limitation: TuSimple has clear highway scenes and few occlusions, so the clean test set alone cannot show whether our changes help under occlusion. We therefore (i) report the clean official score so results stay comparable, (ii) add a fixed synthetic occlusion test on every test frame at several masking levels, and (iii) report a small subset of naturally occluded frames that we tag before seeing any model output (Section 13.4). Synthetic masks are only a proxy for real vehicles, and we say so in the report. Hand-picked pictures are used only as illustration.

7. Dataset Size Feasibility

The raw TuSimple download is about 20GB because it holds full 20-frame clips, but only the 20th frame of each clip is labeled. We use the **Preprocessed Simplified-TuSimple dataset on Kaggle** (https://www.kaggle.com/datasets/rangalamahesh/preprocessed-1), which keeps only the 6,408 labeled frames. This is under **2 GB**, well under the 12GB recommended limit, so it downloads fast and trains on a local machine or Google Colab. The occlusion masks are drawn on these full-resolution labeled frames using their ground-truth lanes, so no extra data is needed.

CLRNet expects the original TuSimple folder layout: a `clips/` folder, the label JSON files and the test label file. It also needs segmentation labels made with the repository's `tools/generate_seg_tusimple.py`. We will check that the Kaggle version matches this layout and fix it if not.

8. Model Survey

We use CLRNet (2022) as our baseline. Two models were published after it. Table 2 compares them.

Model	Year	Main idea	Public code	TuSimple Acc / F1 (%)
CLRNet [1] (baseline)	2022	Line priors refined from high-level to low-level features; ROIGather adds context; Line IoU loss.	PyTorch, with weights	**96.84 / 97.89**
CLRerNet [2]	2024	Same design as CLRNet, with LaneIoU used for sample assignment and loss to give better confidence scores.	PyTorch, CULane weights only	not reported
Lane2Seq [3]	2024	A transformer that writes each lane as a sequence of tokens.	none found	— / 97.95

Table 2: CLRNet and the models published after it. CLRNet numbers are from its paper (ResNet-18); Lane2Seq's number is from its own paper.

Code and weights. CLRNet: https://github.com/Turoad/CLRNet (weights on its Releases page). CLRerNet: https://github.com/hirotomusiker/CLRerNet. We found no public code for Lane2Seq. CLRNet was trained on one GPU for 70 epochs on TuSimple [1].

Why we chose CLRNet:

- It has public PyTorch code and pretrained TuSimple weights, so we can first reproduce its published numbers.
- ROIGather is a small block that is easy to change and test.
- The ResNet-18 version is small (11.9 GFLOPs) and already reaches 97.89% F1, so we can train it several times on one GPU.

Why not CLRerNet: Its code, weights and results are for CULane (and CurveLanes), not TuSimple, so we have no TuSimple numbers to check against.

Why not Lane2Seq: It is only 0.06 F1 higher, uses a different design with no ROIGather-like block to modify, and has no public code that we could find.

9. Baseline Model Architecture

CLRNet [1] resizes the image to 320×800 and extracts features with a ResNet-18 backbone [7] and a Feature Pyramid Network (FPN) [8]. This gives three feature levels $\{L_0, L_1, L_2\}$.

Lane prior. A lane is $N = 72$ points at evenly spaced rows. Each prior has a lane score, a start point, an angle, a length and N horizontal offsets.

Cross-layer refinement. Lane priors are refined three times: first on L_0 (most semantic), then L_1, then L_2 (most detailed). Each step gets the ROI features of the current priors and predicts better priors.

ROIGather (the part we change). For each prior, ROIAlign [6] samples $N_p = 36$ points along it, giving $\mathcal{X}_p \in \mathbb{R}^{C \times N_p}$ with $C = 64$. For L_1 and L_2, ROI features from earlier levels are added. Convolutions along the prior let each point use its neighbors, and a fully connected layer reduces the result to one vector $\mathcal{X}_p \in \mathbb{R}^{C \times 1}$. The feature map is resized to 10×25 and flattened to $\mathcal{X}_f \in \mathbb{R}^{C \times HW}$. Cross-attention is

$$\mathcal{W} = \text{softmax}\left(\frac{\mathcal{X}_p^T \mathcal{X}_f}{\sqrt{C}}\right), \qquad \mathcal{G} = \mathcal{W}\mathcal{X}_f^T,$$

and $\mathcal{G}$ is added to $\mathcal{X}_p$.

Uniform weighting. Before the fully connected layer, nothing decides per input which of the 36 points to trust: a point on a clear marking, on a vehicle or in a shadow enters the layer in the same way. We call this uniform weighting. It is the baseline for Section 12.1.

Loss. Focal loss for the lane score, smooth-ℓ_1 loss for start point, angle and length, and Line IoU loss $\mathcal{L}_{LIoU} = 1 - \text{LIoU}$ for the lane shape.

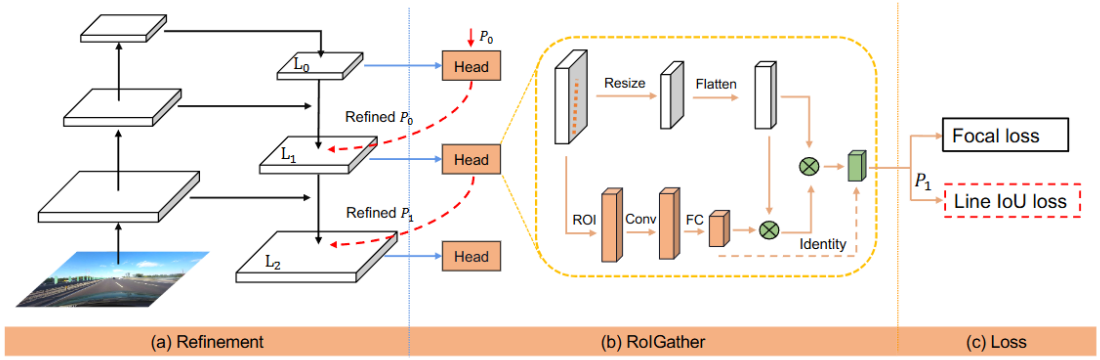


Figure 2. Overview of the proposed **CLRNet**. (a) The network generates feature maps from FPN [9] structure. Subsequently, each lane prior will be refined from high-level features to low-level features. (b) Each head will exploit more contextual information for lane prior features. (c) Classification and regression of lane priors. The proposed Line IoU loss helps further improve the regression performance.

Figure 3: CLRNet, from Figure 2 of [1]: (a) cross-layer refinement, (b) the original ROIGather module, (c) classification and Line IoU loss.

10. Baseline Source Code

Repository URL: https://github.com/Turoad/CLRNet
Written in PyTorch, with configs for TuSimple, CULane and LLAMAS. Training and testing run through `main.py`, for example `python main.py configs/clrnet/clr_resnet18_tusimple.py --gpus 0`. The repository lists Python 3.8, PyTorch 1.6/1.8 and CUDA 10.x as tested, so we will use a matching setup. We change four things: the ROIGather module (gate and GMP-ECA), the config, the data pipeline (a transform that paints the occlusion mask and returns a clean and a masked view), and the loss (the consistency term). The backbone, FPN, priors and the original losses are not changed.

11. Model Weights URL

Weights: https://github.com/Turoad/CLRNet/releases
The README links its checkpoints there, including ResNet-18 for TuSimple (F1 97.89). We use it to check our evaluation setup (Section 13.2) with `--load_from`. The released checkpoint cannot be reused for variants that add new modules, so for the ablation we retrain the baseline and every variant from the same starting point (the ImageNet-pretrained backbone used by the repository config) with the same settings, so the comparison is fair (Section 13).

12. Proposed Architectural and Training Updates

We make three changes. The first two are inside ROIGather and the third is a training loss. Each one can be switched on or off in the config, so we can test them one at a time and together (Section 13).

12.1 Visibility-aware point gating (new)

Update: For each lane prior, a small gate predicts one reliability weight for each of the $N_p = 36$ sampled positions. The ROI feature is multiplied by these weights *before* any pooling and before cross-attention, so positions that fall on a vehicle, in a shadow or in a gap of a dashed line can be turned down. The gate goes after the convolutions along the prior and before GMP-ECA and the fully connected layer.

$$\mathbf{g} = \sigma\big(\mathbf{w}_2^\top \,\mathrm{ReLU}(W_1 \mathcal{X}_p + \mathbf{b}_1) + b_2\big) \in (0,1)^{N_p}, \qquad \hat{\mathcal{X}}_p = \mathcal{X}_p \odot \mathbf{g}.$$

Here $W_1 \in \mathbb{R}^{(C/4)\times C}$ and $\mathbf{w}_2 \in \mathbb{R}^{C/4}$ are applied at every position with the same weights (two 1×1 convolutions along the point axis), and $\mathbf{g}$ is applied to all C channels. There is one gate per refinement step. Each gate has 1,057 weights (3,171 in total).

Design notes:

- **Context:** The convolutions before the gate already mix neighboring points, so the feature at each position carries some information about its neighbors.
- **No visibility labels:** TuSimple has none, so the gate is trained end to end from the detection loss. The occlusion training in Section 12.3 gives it examples in which we know which points are hidden, but we do not supervise it directly.
- **Start near the baseline:** The last bias is set so that $\mathbf{g} \approx 0.95$ at the start of training.
- **Why gating before max pooling can work:** We expect the convolutions in the repository to end in a ReLU, so $\mathcal{X}_p \geq 0$ and a weight below 1 can only shrink a response. We will verify this in the code. If it is not true, we will gate after a ReLU and document it.
- **What can go wrong:** The gate can collapse to a constant, or can change the score without tracking visibility. We check this directly (Section 13.5).

12.2 Max-pooled efficient channel attention (GMP-ECA)

Update: We add a **1D Max-Pooled Efficient Channel Attention block (GMP-ECA)** inside ROIGather, after the gate and before the fully connected layer, so it works on $\hat{\mathcal{X}}_p \in \mathbb{R}^{C\times N_p}$ ($C = 64$, $N_p = 36$). There is one block per refinement step (three in total).

How it works:

$$\mathbf{z} = \max_{n=1,\ldots,N_p} \hat{\mathcal{X}}_p[:, n] \in \mathbb{R}^C, \qquad \mathbf{s} = \sigma\big(\mathrm{Conv1D}_{k=3}(\mathbf{z})\big), \qquad \tilde{\mathcal{X}}_p = \hat{\mathcal{X}}_p \odot \mathbf{s} + \hat{\mathcal{X}}_p.$$

The 1D convolution runs across the channels, $\mathbf{s}$ is applied to all N_p points, and the last term is a shortcut. The kernel size $k = 3$ follows the ECA-Net rule [4] for $C = 64$. Each block adds only 3 weights (9 in total).

Four ROIGather variants. If the gate is off, $\mathbf{g} = \mathbf{1}$. If GMP-ECA is off, the block is removed and $\tilde{\mathcal{X}}_p = \hat{\mathcal{X}}_p$. This gives: (i) uniform weighting (the baseline), (ii) gate only, (iii) GMP-ECA only, and (iv) gate and GMP-ECA together. Average-pooled ECA is a fifth variant, used as a control.

Why we think it may help (a guess we will test):

- **Missing in the baseline:** After the convolutions, every lane prior goes through the same fully connected layer. Nothing reweights channels based on that prior's content. Channel attention does this at almost no cost [4].
- **Why max instead of average pooling:** Standard ECA averages each channel over all points. Along a lane prior, some of the 36 points fall in gaps of dashed lines, on a vehicle or in shadow. Averaging mixes these in, while max pooling keeps the strongest response and so depends on the visible part of the lane. Max pooling could also boost noise on priors with no lane. The occlusion curves of max-pooled and average-pooled ECA (Section 13.4) are the direct test of this claim.
- **How it fits with the gate:** When the gate comes first, points with low reliability contribute little to the maximum. The two parts are meant to work together, but they might also overlap or interfere, so we test them alone and together.
- **What it does not do:** It does not guess hidden lane shape. That is still done by ROIGather's convolutions and cross-attention. The block only changes which channels are stressed before cross-attention.
- **Expected gain:** TuSimple is almost saturated (Section 4), so on the clean test set we expect at most a few tenths of an F1 point. The more interesting question is the occlusion curve. We judge every result against the variation between random seeds, not a single run, and we report negative results as they are.

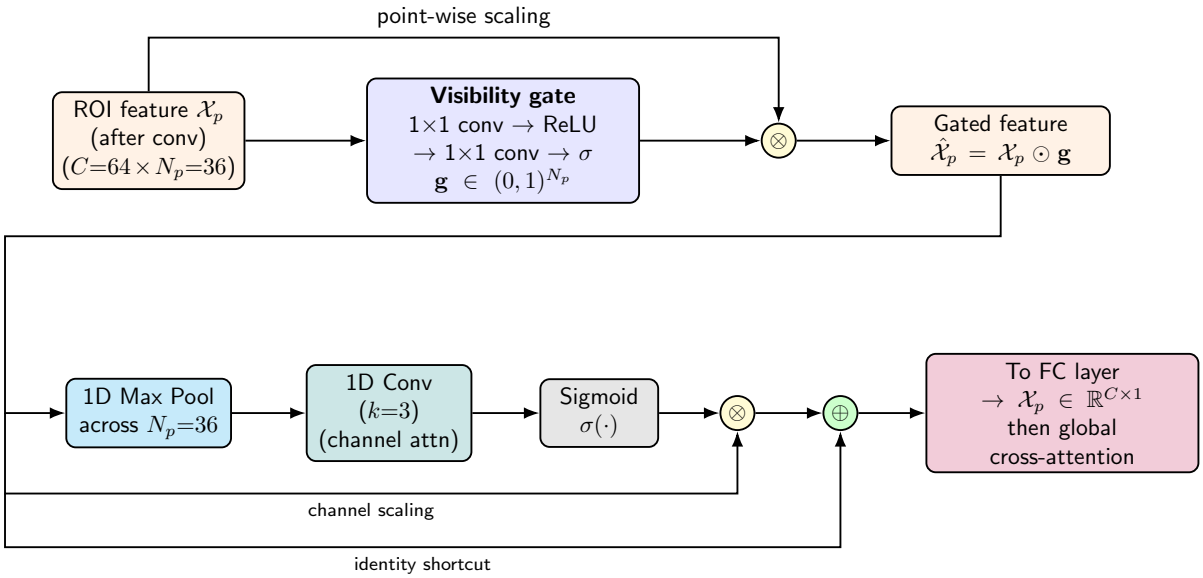


Figure 4: Proposed ROIGather update. Top: the visibility gate rescales each of the 36 sampled points. Bottom: max-pooled ECA on the gated feature, between the convolutions and the fully connected layer that feeds cross-attention.

12.3 Occlusion-consistency training (new)

Update: During training we make a second view of each image in which part of one annotated lane is hidden by a controlled mask. We then ask the network to give the same lane confidence and coordinates on the masked view as on the clean view, using the clean prediction as a fixed (detached) target.

Controlled mask. The mask is made after the usual CLRNet augmentation, so the lane labels still match the image. It is built like this:

1. Pick one annotated lane at random.
2. Pick a contiguous range of rows along that lane that covers a fraction m of its annotated vertical extent, at a random position. In training, $m \sim U(0.1, 0.5)$.
3. Paint a band over the lane in those rows. At each row the half-width of the band is 0.3 times the distance to the nearest neighboring lane (with a small minimum so the marking is always covered), so the band hides the target lane but not its neighbors.
4. Fill the band with a random neutral color plus light pixel noise.

These values are first choices. We fix them before the final runs and never change them after seeing test results. Cutout [9] and Random Erasing [10] also hide image regions, but at random places. Ours hides a chosen part of an annotated lane by a known amount, which lets us measure robustness by masking level.

Two views. Each training image gives a clean view I and a masked view $\tilde{I}$ with the same ground truth. Both pass through the same network in one batch, and both get the standard CLRNet loss against the ground truth (each view is matched to the labels on its own).

Consistency loss. CLRNet's priors are a fixed learnable set, so prior i in the clean view is the same prior as prior i in the masked view, and no matching step is needed. At refinement step t, let p_i^t be the lane score and $\mathbf{r}_i^t$ the regression output (start point, angle, length, offsets) of prior i. A tilde marks the masked view and $\mathrm{sg}(\cdot)$ is stop-gradient:

$$\mathcal{L}_{cons} = \frac{1}{T}\sum_{t=1}^{T}\left[\frac{1}{M}\sum_{i=1}^{M}\mathrm{BCE}\big(\tilde{p}_i^t,\, \mathrm{sg}(p_i^t)\big) + \frac{1}{|\mathcal{P}^t|}\sum_{i\in\mathcal{P}^t}\text{smooth-}\ell_1\big(\tilde{\mathbf{r}}_i^t,\, \mathrm{sg}(\mathbf{r}_i^t)\big)\right].$$

Here $T = 3$ steps, M is the number of priors, and $\mathcal{P}^t$ is the set of priors assigned to a ground-truth lane in the clean view at step t. The total loss is

$$\mathcal{L} = \mathcal{L}_{sup}(I) + \mathcal{L}_{sup}(\tilde{I}) + \lambda(e)\,\mathcal{L}_{cons}, \qquad \lambda(e) = \lambda_{max}\cdot\min(1, e/10),$$

where e is the epoch. We fix $\lambda_{max} = 1$ in advance and do not tune it on test data. The ramp is there because clean predictions are poor early in training.

What this can and cannot give. The masked view is already supervised by the ground truth, so matching the clean prediction only helps if it adds information beyond that. It may not, and it may also smooth predictions and lower clean F1. This is why every compared model gets exactly the same two-view training and the only difference is the consistency term (Section 13.1).

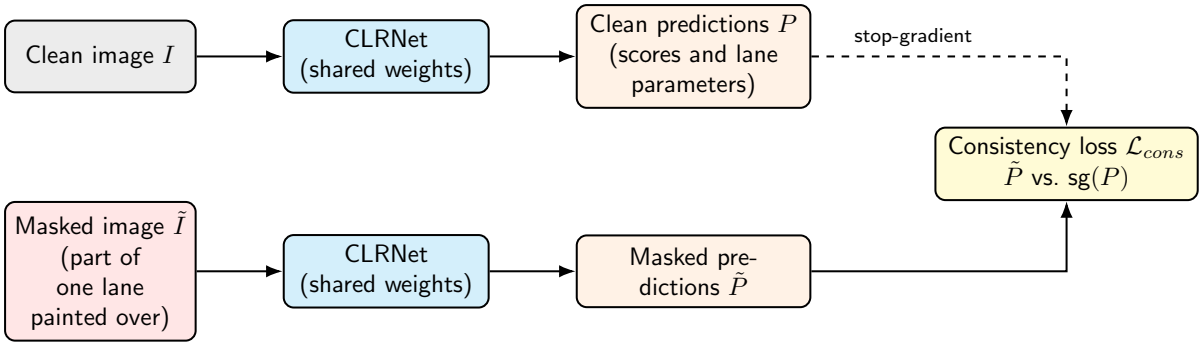


Figure 5: Occlusion-consistency training. Both views also receive the standard CLRNet loss against the same ground truth. The clean predictions are a detached target for the masked ones.

13. Full Evaluation Plan

13.1 Models compared

Table 3 lists the models. All rows R1–R7 use the same two-view data (same masks, same seeds), the same schedule and the same augmentation. They differ only in the columns marked. R0 is the original single-view recipe and shows what the occlusion augmentation itself does to clean performance.

ID	Model	Gate	GMP-ECA	Avg-ECA	Occ. aug.	Consist.	Seeds
R0	Original CLRNet recipe (reference)	—	—	—	—	—	2–3
R1	Baseline CLRNet, uniform weighting	—	—	—	✓	—	3
R2	R1 + visibility gate	✓	—	—	✓	—	3
R3	R1 + max-pooled ECA	—	✓	—	✓	—	3
R4	R1 + gate + max-pooled ECA	✓	✓	—	✓	—	3
R5	R1 + average-pooled ECA (control)	—	—	✓	✓	—	2–3
R6	R1 + consistency loss	—	—	—	✓	✓	3
R7	Best of R2–R4 + consistency loss		as selected		✓	✓	3

Table 3: Models in the study. "Occ. aug." means two-view training with the controlled mask, with the standard loss on both views.

Comparisons that answer each question:

- **Weighting and attention:** R1 (uniform) vs. R2 (gate only) vs. R3 (GMP-ECA only) vs. R4 (both). R3 vs. R5 tests max against average pooling.
- **Consistency loss:** R1 vs. R6, and the selected variant vs. R7. Both sides of each pair have the same occlusion augmentation, so a difference can be attributed to the consistency term.
- **Augmentation itself:** R0 vs. R1.

The best of R2–R4 for R7 is chosen by the mean F1 over masking levels 10–50% on the masked *validation* frames. Test labels are not used. If none of R2–R4 beats R1 on validation, R7 is dropped because it would equal R6.

13.2 Training protocol and checkpoints

All models use the same settings (70 epochs, same split and augmentation, hyperparameters from the repository config). We train on the 3,268 train frames and monitor on the 358 validation frames. We will check whether the repository's TuSimple config validates on the test split. If it does, we change it so that no checkpoint is chosen with test labels. Every model uses its final-epoch checkpoint, and the test set is evaluated once per trained model. Because we train on the train split only, our baseline may differ a little from the published number, so we judge each change against our own baseline R1.

Pipeline check: before any training, we run the released ResNet-18 TuSimple checkpoint through our setup with `--load_from` on the clean test set and confirm that it gives about the published 97.89 F1. This checks the data layout and the evaluation script.

13.3 Clean evaluation and decision rule

Metrics: Accuracy, F1 Score, False Positives (FP), and False Negatives (FN) on the official, unmodified TuSimple test split.

Each model is trained with three random seeds (two for R0 and R5 if Colab limits require it). We report mean and standard deviation. We call a change an improvement only if the mean gain is larger than the larger of the two standard deviations *and* the gain has the same sign for every pair of runs with the same seed number. With three seeds this is a consistency check and not a significance test, and we say so in the report. We also check that no change lowers clean F1 by more than one standard deviation.

13.4 Occlusion robustness protocol (fixed before testing)

We evaluate robustness directly. Everything below is written into code and fixed before any trained model is run on the test set.

1. **Images:** all 2,782 test frames. We do not choose frames, so there is no selection effect. The 358 validation frames get the same treatment (with a different seed) and are used for model selection.

2. **Which lane:** for each frame, one lane is chosen by a seeded random draw among lanes with at least 20 annotated points. If no lane qualifies, the frame stays unmasked at every level and is left out of the masked-lane metrics (we report how many).

3. **Corruption:** the band mask of Section 12.3, drawn on the original 1280×720 frame with the ground-truth lane, at a fixed masking level m. The position along the lane comes from the same seeded generator. The ground-truth labels are not changed.

4. **Levels:** $m \in \{0, 10, 20, 30, 40, 50\}\%$, which lie inside the training range, plus 70%, which lies outside it. Level 0 is the original test set, so it equals the standard clean result.

5. **Frozen test set:** all masked frames are generated once from a fixed seed, saved with a checksum and a manifest, and every model is evaluated on exactly these images. The test seeds are different from the training seeds.

6. **Shifted corruption (secondary):** the same masks with a different fill (uniform random noise instead of a neutral color), to check that models have not just learned the look of the training mask.

7. **Metrics at each level:** official Accuracy, F1, FP and FN, plus masked-lane recall and masked-segment accuracy (Section 3). The **primary robustness metric** is masked-lane recall averaged over levels 10–50%. The decision rule of Section 13.3 is applied to it.

8. **Natural-occlusion subset (secondary):** before any model prediction is viewed, one team member tags test frames in which a real vehicle overlaps a lane marking, using only the image and its ground truth. The tag file is committed and frozen. We report F1 on this subset together with its size. If it is small, we treat the result as indicative only.

13.5 Mechanism check for the gate

For the gate models (R2, R4) on the masked test frames, we take the priors that detect the masked lane and split their 36 sampled points into points inside the mask and points outside it. We report the mean gate value in each group, and a histogram of all gate values to check for collapse. If the gate is not lower inside the mask, then it is not acting as a visibility detector, and we say so even if F1 changes.

13.6 Planned plots for the final report

- **Training and Validation Curves:** loss and validation F1 per epoch, averaged over seeds.
- **Occlusion Robustness Curves:** F1 and masked-lane recall against masking level (0–70%) for R1–R7, mean ± standard deviation over seeds, for the main and the shifted fill.
- **Ablation Table and Bar Charts:** clean Accuracy and F1 (mean ± standard deviation) for all models, plus the robustness summary.
- **Gate Diagnostics:** mean gate value inside and outside the mask, and the gate histogram.
- **Qualitative Comparison Grids:** Ground Truth, Baseline and Proposed predictions side by side. One grid uses frames drawn with a fixed seed. A second grid shows the largest wins and losses and is labeled as chosen after the fact. Vehicle, dashed-marking and shadow frames from the natural-occlusion tags are added as illustration.

13.7 Compute plan

Two-view training costs roughly twice as much per step as normal training. The table has 6 rows with 3 seeds and 2 rows with 2–3 seeds, so 22–24 runs, against 9 in the earlier plan. We time the first baseline run and decide then. If compute runs short we keep the same rules for all models and cut in this order: seeds of R0 and R5 (down to two), then the seeds of the remaining rows (down to two). We run in the order R1, R3, R2, R4, R6, R7, R5, R0. R7 has to wait for R2–R4 because it depends on the validation selection.

14. Graduation Project / Thesis Statement

Neither team member has commenced a graduation project or thesis (expected graduation: Fall 2027 & Spring 2028).

15. Related Prior Projects

Mahmoud Afifi:

- Face Recognition Authentication System using deep convolutional networks.
- Q-Learning Maze Pathfinder.

- Credit Card Default Prediction.

Ziad Eliwa:

- Customer Arabic Sentiment Analysis with Fine-Tuning AraBERT
- Network Intrusion Detection using MLPs

17. Technical Work Distribution

- **Mahmoud Afifi:** Environment and Kaggle dataset setup (including checking the folder layout CLRNet expects); running baseline evaluation and the pipeline check; the occlusion mask generator, the frozen masked validation and test sets with manifest and checksums, and the two-view data transform; metric extraction scripts, including masked-lane recall and masked-segment accuracy; training runs R0, R1, R5 and R6.
- **Ziad Eliwa:** PyTorch implementation of the visibility gate and the 1D Max-Pooled ECA block inside the ROIGather module; the consistency loss and the two-view training loop; training runs R2, R3, R4 and R7; training curves, robustness curves, gate diagnostics and qualitative prediction plots.
- **Both:** tagging of the natural-occlusion subset (one person tags, the other checks the tag file is frozen before predictions are viewed) and writing the final report.

References

[1] Zheng, T., Huang, Y., Liu, Y., Tang, W., Yang, Z., Cai, D., & He, X. (2022). *CLRNet: Cross Layer Refinement Network for Lane Detection*. Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR), pp. 898–907.

[2] Honda, H., & Uchida, Y. (2024). *CLRerNet: Improving Confidence of Lane Detection with LaneIoU*. Proceedings of the IEEE/CVF Winter Conference on Applications of Computer Vision (WACV), pp. 1176–1185.

[3] Zhou, K. (2024). *Lane2Seq: Towards Unified Lane Detection via Sequence Generation*. Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR), pp. 16944–16953.

[4] Wang, Q., Wu, B., Zhu, P., Li, P., Zuo, W., & Hu, Q. (2020). *ECA-Net: Efficient Channel Attention for Deep Convolutional Neural Networks*. Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR).

[5] TuSimple. (2017). *TuSimple Lane Detection Benchmark*. https://github.com/TuSimple/tusimple-benchmark.

[6] He, K., Gkioxari, G., Dollár, P., & Girshick, R. (2017). *Mask R-CNN*. IEEE International Conference on Computer Vision (ICCV), pp. 2961–2969.

[7] He, K., Zhang, X., Ren, S., & Sun, J. (2016). *Deep residual learning for image recognition*. IEEE Conference on Computer Vision and Pattern Recognition (CVPR), pp. 770–778.

[8] Lin, T.-Y., Dollár, P., Girshick, R., He, K., Hariharan, B., & Belongie, S. (2017). *Feature pyramid networks for object detection*. IEEE Conference on Computer Vision and Pattern Recognition (CVPR), pp. 2117–2125.

[9] DeVries, T., & Taylor, G. W. (2017). *Improved regularization of convolutional neural networks with Cutout*. arXiv:1708.04552.

[10] Zhong, Z., Zheng, L., Kang, G., Li, S., & Yang, Y. (2020). *Random erasing data augmentation*. Proceedings of the AAAI Conference on Artificial Intelligence, pp. 13001–13008.

Other sources: the CLRNet repository (https://github.com/Turoad/CLRNet) and the CLRerNet repository (https://github.com/hirotomusiker/CLRerNet).